

Geometric Algorithms

Báo cáo nhóm 14

Phân tích và thiết kế thuật toán(24520002)

Nhóm: Nhóm 14

Thành viên:

Đặng Phú Duy 24520010

Mai Quốc Anh 24520002

Ngày 19 tháng 11 năm 2025

1 Tóm tắt đề bài

Bản đồ thế giới được mô phỏng như một mặt phẳng hình chữ nhật có kích thước $W \times H$. Trên bản đồ tồn tại hai loại đối tượng chính:

- **Các vùng theo dõi Aura** (gồm n vùng): Mỗi vùng được biểu diễn bằng một hình chữ nhật với hai điểm:
 - (X_i, Y_i) : góc trái dưới,
 - (A_i, B_i) : góc phải trên.

Các vùng không giao nhau và không chạm biên bản đồ.

- **Các Hunter** (gồm m người): Hunter thứ j có:
 - tọa độ (C_j, R_j) ,
 - lượng Aura t_j .

Nhiệm vụ của hệ thống giám sát Aura là thu thập dữ liệu từ tất cả các Hunter và xác định lượng Aura được ghi nhận trong các vùng theo dõi.

Yêu cầu

1. Với mỗi vùng theo dõi Aura i , tính tổng lượng Aura của tất cả Hunter nằm bên trong vùng đó.
2. Tính tổng lượng Aura của các Hunter không thuộc bất kỳ vùng theo dõi nào.

Kết quả xuất ra

In ra tổng cộng $n + 1$ số nguyên:

- Số thứ i ($1 \leq i \leq n$): tổng Aura trong vùng theo dõi i ,
- Số thứ $n + 1$: tổng Aura nằm ngoài tất cả vùng theo dõi.

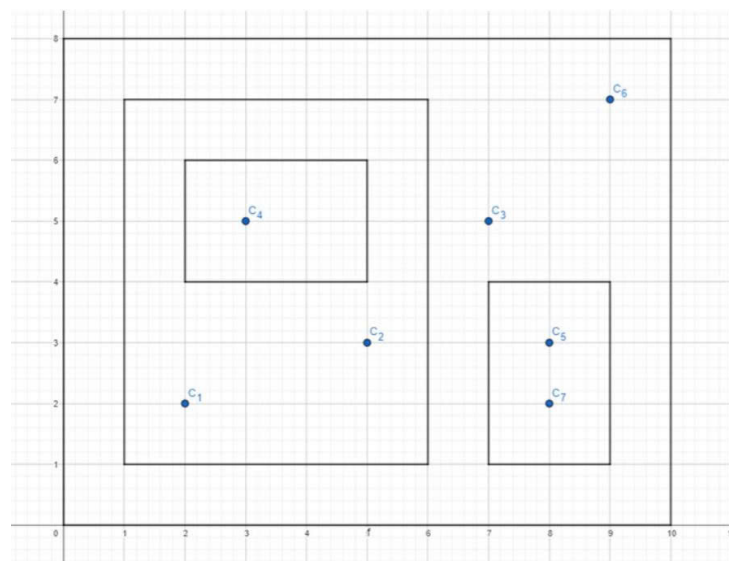
Sample Input

```
10 8 3 7
1 1 6 7
7 1 9 4
2 4 5 6
2 2 7
5 3 9
```

7 5 15
 3 5 4
 8 3 7
 9 7 5
 8 2 10

Sample Output

16 17 4 20



Hình 1: Mô tả hình ảnh

2 Lời giải chi tiết

2.1 Subtask 1

2.1.1 Phương pháp thiết kế

Trong subtask 1, ta có n, m nhỏ ($n, m \leq 10^3$), không gian nghiệm nhỏ. -> Tập hợp các trường hợp cần duyệt có kích thước đủ nhỏ để duyệt hết trong thời gian giới hạn.

Kiểm tra một Hunter có nằm trong vùng hay không chỉ cần 4 phép so sánh. Thời gian mỗi lần kiểm tra = $O(1)$, cực kỳ rẻ. -> Kiểm tra điều kiện rất dễ

Vì vậy, ta có thể sử dụng phương pháp thiết kế thuật toán cơ bản nhất: **Brute Force**.

2.1.2 Thuật toán chi tiết

Với Subtask 1, các giới hạn dữ liệu nhỏ cho phép áp dụng phương pháp **Brute Force** (duyệt toàn bộ). Ý tưởng chính của thuật toán được mô tả như sau:

1. **Duyệt từng Hunter:** Với mỗi Hunter, ta lần lượt xét qua tất cả các vùng theo dõi có trong bản đồ.
2. **Kiểm tra thuộc vùng:** Duyệt và tìm vùng có diện tích nhỏ nhất thỏa điều kiện là có tọa độ của hunter trong vùng đó. Một Hunter được xem là nằm trong vùng i khi thỏa mãn điều kiện:

$$X_i \leq C_j < A_i, \quad Y_i \leq R_j < B_i$$

trong đó (X_i, Y_i) là góc trái dưới, (A_i, B_i) là góc phải trên của vùng i .

3. Cập nhật tổng Aura:

- Nếu Hunter nằm trong một vùng, cộng lượng Aura t_j của Hunter vào tổng Aura của vùng đó và **dừng kiểm tra các vùng tiếp theo**, vì mỗi Hunter chỉ được ghi nhận vào **một vùng duy nhất** (vùng nhỏ nhất chứa nó theo yêu cầu đề bài).
 - Nếu Hunter không thuộc bất kỳ vùng nào, cộng lượng Aura của Hunter vào biến **outside**, tức là tổng lượng Aura không bị theo dõi.
4. **Lặp lại cho tất cả các Hunter:** Thực hiện quy trình trên cho tất cả m Hunter trong bản đồ.

2.1.3 Độ phức tạp

1. Độ phức tạp thời gian

Thuật toán Brute Force thực hiện các bước sau:

1. Với mỗi Hunter (tổng cộng m Hunter), ta duyệt qua tất cả n vùng theo dõi.
2. Với mỗi lần kiểm tra một Hunter với một vùng, chỉ thực hiện **4 phép so sánh** để kiểm tra Hunter có nằm trong hình chữ nhật hay không.

Do đó, tổng số phép toán trong trường hợp xấu nhất là:

$$O(n \cdot m)$$

2. Độ phức tạp không gian

Thuật toán sử dụng các cấu trúc dữ liệu sau:

- Mảng `rect[n]` lưu thông tin n vùng theo dõi $\Rightarrow O(n)$.
- Mảng `ans[n]` lưu tổng Aura cho từng vùng $\Rightarrow O(n)$.
- Biến `outside` lưu tổng Aura không thuộc vùng nào $\Rightarrow O(1)$.

- Các biến phụ trợ (như tọa độ tạm thời, biến `found`) $\Rightarrow O(1)$.

Vì các Hunter được xử lý lần lượt, không cần lưu trữ toàn bộ, tổng bộ nhớ sử dụng là:

$$O(n)$$

2.2 Subtask 2

2.2.1 Phương pháp thiết kế

Trong subtask2, ta có $W, H \leq 10^3$, không gian tọa độ nhỏ $\rightarrow$ Tập các điểm duyệt nhỏ

Không sử dụng cấu trúc dữ liệu nâng cao.

Quyết định cho mỗi ô dựa trên kiểm tra tất cả vùng nhỏ hơn trước đó

Vì vậy, ta có thể sử dụng phương pháp thiết kế thuật toán : **Brute Force**.

2.2.2 Thuật toán chi tiết

1. Khởi tạo:

- Tạo mảng 2D `grid[W][H]` với giá trị `-1` (chưa có vùng nào).
- Tạo mảng `zone_aura[n]` lưu tổng Aura từng vùng.
- Khởi tạo biến `untracked = 0` lưu Aura ngoài vùng theo dõi.

2. Sắp xếp vùng:

- Tính diện tích từng vùng: $(A_i - X_i) \times (B_i - Y_i)$.
- Sắp xếp các vùng theo diện tích tăng dần.

3. Gán tọa độ cho vùng:

- Duyệt các vùng theo thứ tự đã sắp xếp.
- Với mỗi ô (x, y) trong vùng, nếu `grid[x][y] == -1`, gán `grid[x][y] = index_vung`.
- Ta có thể dùng cấu trúc dữ liệu để tối ưu chỉ duyệt qua những tọa độ có `grid[x][y] == -1`

4. Cộng Aura Hunter:

- Với mỗi Hunter (C_j, R_j, t_j) :
 - Nếu `grid[Cj][Rj] != -1` $\rightarrow$ cộng t_j cho vùng tương ứng.
 - Nếu `grid[Cj][Rj] == -1` $\rightarrow$ cộng t_j vào `untracked`.

5. Xuất kết quả:

- In tổng Aura từng vùng và `untracked`.

2.2.3 Phân tích độ phức tạp

Độ phức tạp không gian (Space Complexity)

Thuật toán sử dụng các cấu trúc dữ liệu chính sau:

- Mảng 2D `grid[W][H]` lưu chỉ số vùng của mỗi ô: $O(W \cdot H)$.
- Mảng `zone_aura[n]` lưu tổng Aura từng vùng: $O(n)$.
- Biến `untracked` lưu Aura ngoài vùng: $O(1)$.

Do đó, tổng không gian yêu cầu là:

$$O(W \cdot H + n)$$

Độ phức tạp thời gian (Time Complexity) Các bước chính:

1. Sắp xếp n vùng theo diện tích: $O(n \log n)$.
2. Gán tọa độ cho các ô trong vùng:
 - Mỗi ô trong bản đồ chỉ được gán một lần.
 - Thời gian tổng: $O(W \cdot H)$.
3. Cộng Aura cho m Hunter: $O(m)$.
4. Xuất kết quả: $O(n)$.

Do đó, tổng thời gian chạy của thuật toán là:

$$O(n \log n + W \cdot H + m)$$

2.3 Subtask 3

2.3.1 Phương pháp thiết kế

Vì thuật chính của bài này là sweepline, nên phương pháp tiếp cận của thuật toán này là **Transforms**.

Cụ thể, bài toán được biến đổi từ bài toán 2D xuống thành bài toán 1D nhờ vào việc duyệt sweepline, Xử lý theo thứ tự tuyến tính, các sự kiện (bắt đầu/kết thúc vùng, Hunter) được sắp xếp theo x .

2.3.2 Thuật toán chi tiết

Ta sẽ thực hiện cách tiếp cận transforms, sử dụng thuật toán sweepline để giải quyết bài toán.

1. Tạo sự kiện:

- Với mỗi vùng theo dõi: tạo 2 sự kiện x_1 (bắt đầu) và x_2 (kết thúc).
- Với mỗi Hunter: tạo sự kiện tại C_x .

2. Sắp xếp sự kiện theo trục x tăng dần.

3. Duyệt các sự kiện:

- **Bắt đầu vùng:** thêm đoạn y của vùng vào danh sách active.
- **Kết thúc vùng:** xóa đoạn y khỏi danh sách active.
- **Hunter:** kiểm tra đoạn active chứa $C_y \rightarrow$ cộng Aura vào vùng đó; nếu không có, cộng vào tổng Aura ngoài vùng.

4. Kết quả: tổng Aura cho từng vùng và tổng Aura ngoài vùng.

2.3.3 Phân tích độ phức tạp

Độ phức tạp thời gian:

- Tạo sự kiện: mỗi vùng tạo 2 sự kiện, mỗi Hunter tạo 1 sự kiện. Tổng số sự kiện là $2n + m = O(n + m)$, thời gian tạo sự kiện là $O(n + m)$.
- Sắp xếp sự kiện theo trục x : $O((n + m) \log(n + m))$.
- Duyệt sự kiện:
 - Bắt đầu/kết thúc vùng: thêm/xóa đoạn y vào danh sách active $\Rightarrow O(1)$ nếu danh sách đơn giản (vùng không chồng nhau).
 - Hunter: kiểm tra đoạn active chứa $C_y \Rightarrow O(\log n)$ nếu dùng binary search.

Tổng thời gian:

$$O((n + m) \log n)$$

Độ phức tạp không gian:

- Danh sách sự kiện: $O(n + m)$
- Danh sách các đoạn y active: tối đa n đoạn $\Rightarrow O(n)$
- Các biến khác (tổng Aura): $O(n)$

Tổng không gian:

$$O(n + m)$$